

Language Models

Rasha Obeidat, PhD

AI455: NLP

Rasha Obeidat

Language Models (LMs)

- A **Language Model (LM)** is a probabilistic model that assign probability distribution over sequences of words that matches distribution of a language.
- To specify a correct probability distribution, the probability of all sentences in a language must sum to 1.
- Typically, LMs predict the next word given the previous context.

Language Models Uses

1. Language models are used to **rank or generate the most probable sequences of words** in various tasks, including:
   - **Example:**
     - **In Machine Translation, LM** Prefers translations that form fluent and likely target-language sentences
     - **In Text Generation**: **LM** Produces grammatically and contextually coherent output.

Language Model Uses

2. Text prediction or completion: LMs can be used to **predict the next word(s)** in an incomplete sentence (or sequence).
   - This is useful in applications like **autocomplete** or **predictive typing**, where the system suggests how to continue what you're writing.
     - **Examples:**
       - الرجاء إغلاق الهاتف _____ → **الخلوي**
       - هذا الطفل لا يستطيع _____ → **المشي**

Language Models

- **Early models:**
  - N-gram Models
  - Recurrent Neural Networks based LMs (e.g., RNNs, LSTM, GRU)
- **Transformer-based models:**
  - Encoder-only (e.g., BERT, RoBERTa) – good for understanding and classification
  - Decoder-only (e.g., GPT-2, GPT-3, GPT-4) – good for generation
  - Encoder–Decoder (e.g., T5, BART) – good for translation and summarization

Rasha Obeidat

N-gram LMs Formulas

- An **n-gram language model** estimates the probability of a word based on the previous $n-1$ words:
- **Chain Rule.** Full probability computed with the **chain rule** as follows:

$$p(w_1, w_2, \dots, w_T) = p(w_1)\, p(w_2|w_1)\, p(w_3|w_1, w_2)\ \dots p(w_T|w_1 w_2 \dots w_{T-1})$$

$$= \prod_{i=1}^{T} p(w_i|w_1 \dots w_{i-1})$$

- **N-gram Approximation.** Since $p(w_i|w_1 \dots w_{i-1})$ is hard to compute (due to data sparsity), N-gram approximation is applied:

$$p(w_i|w_1 \dots w_{i-1}) \approx p(w_i|w_{i-n+1} \dots w_{i-1})$$

and

$$\prod_{i=1}^{T} p(w_i|w_1 \dots w_{i-1}) \approx \prod_{i=1}^{T} p(w_i|w_{i-n+1} \dots w_{i-1})$$

Markov assumption

- The **Markov assumption** states that the future state of a system depends only on a limited history of previous states. Specifically, in a *k*th-order Markov model, the next state depends only on the previous *k* states.
- An N-gram model corresponds to a $(N-1)^{th}$**-order** Markov model.

$$p(w_i|w_1 \dots w_{i-1}) \approx p(w_i|w_{i-n+1} \dots w_{i-1})$$

Computing Sentence Probabilities

- Example: Sentence "**I want to eat Mansaf**"
- **Unigram (1-gram) Model:**
  - The sentence "**I want to eat Mansaf**"
  - $P(\text{ I want to eat Mansaf}) = P(\text{ I }) \times P(\text{want}) \times P(\text{to}) \times P(\text{eat}) \times P(\text{Mansaf})$
- **Bigram (2-gram) Model:**
  - The sentence becomes "**<s> I want to eat Mansaf <\s>**"
  - $P(\text{<s> I want to eat Mansaf <\s>}) = P(\text{ I } |\text{ <s> }) \times P(\text{want } |\text{I}) \times P(\text{to } |\text{want}) \times P(\text{ eat}|\text{ to}) \times P(\text{Mansaf}|\text{eat}) \times P(\text{<\s>}|\text{ Mansaf})$
- **Trigram (3-gram) Model:**
  - The sentence becomes "**<s1> <s2> I want to eat Mansaf <\s1><\s2>**"
  - $P(\text{<s1> <s2> I want to eat Mansaf <\s1><\s2>}) = P(\text{ I}|\text{<s1> <s2> }) \times P(\text{want } |\text{ <s1>, I}) \times P(\text{ to}|\text{ I, want}) \times P(\text{eat}|\text{ want, to}) \times P(\text{Mansaf}|\text{to, eat}) \times P(\text{<\s1>}|\text{ eat Mansaf}) \times P(\text{<\s2>}|\text{ Mansaf, <\s1>})$

Estimating Probabilities from Counts

Unigram

- $p(w_i) = \frac{C(w_i)}{N}$
- $C(w_i)$: count of the word w_i in the corpus
- N: Total number of word tokens in the corpus (including repetitions)

Bigram

- $p(w_i|w_{i-1}) = \frac{C(w_{i-1},w_i)}{C(w_{i-1})}$
- $C(w_{i-1}, w_i)$: count of bigram $C(w_{i-1}, w_i)$ in the corpus
- $C(w_{i-1})$: count of the word w_{i-1} the corpus

Trigram

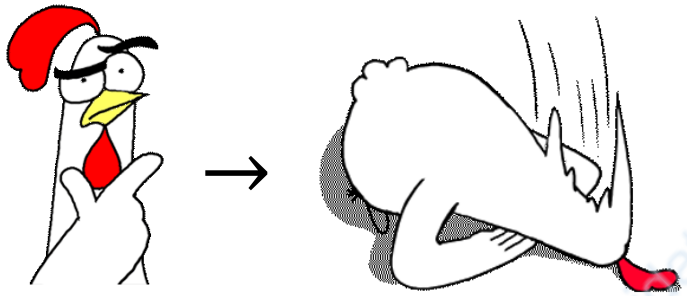
- $p(w_i|w_{i-2}, w_{i-1}) = \frac{C(w_{i-2,} w_{i-1},w_i)}{C(w_{i-2,} w_{i-1})}$
- $C(w_{i-2,} w_{i-1}, w_i)$: count of trigram (w_{i-2}, w_{i-1}, w_i) in the corpus
- $C(w_{i-2}, w_{i-1})$: count of bigram (w_{i-2}, w_{i-1}) in the corpus

Handling Unknown Words

- In real-world scenarios, test data often contains **words not seen during training** called **Out-of-Vocabulary (OOV)** words.
- To address this, we introduce a special token:

<UNK> = unknown word symbol

- Which words in the ***training*** will be considered unknown?
- They are all known!!!!



Handling Unknown Words

Common Strategies:

- **Vocabulary Predefinition**:
  - Before training, define a fixed vocabulary (e.g., most frequent 10k words).
  - All other words are replaced with <UNK>.
- **Single Occurrence Replacement**:
  - Replace the **first occurrence** of each word in the training data with <UNK>.



Rasha Obeidat

Example of Estimating Probabilities for Bigram LM

- **Estimating Probabilities of the bigrams:**

$$P(I \mid < s >) = \frac{100}{500} = 0.2$$

$$P(want \mid I) = \frac{80}{200} = 0.4$$

$$P(to \mid want) = \frac{50}{100} = 0.5$$

$$P(eat \mid to) = \frac{30}{90} \approx 0.333$$

$$P(Mansaf \mid eat) = \frac{10}{40} = 0.25$$

$$P(< /s > \mid Mansaf) = \frac{5}{20} = 0.25$$

Bigram	Count	Unigram	Count
(<s>, I)	100	<s>	500
(I, want)	80	I	200
(want, to)	50	want	100
(to, eat)	30	to	90
(eat, Mansaf)	10	eat	40
(Mansaf, </s>)	5	Mansaf	20

Table: related unigram and bigram occurrences from the corpus

- **Estimating Probabilities of the entire sentence:**

P(<s> I want to eat Mansaf <\s>) =

P(I | <s>) × P(want |I) × P(to |want) × P(eat| to) × P(Mansaf|eat) × P(<\s>| Mansaf) =

$0.2 \times 0.4 \times 0.5 \times 0.333 \times 0.25 \times 0.25 \approx 0.000416$

Limitations of N-gram LMs

1. **Data sparsity** (too many unseen N-grams)
   - The number of possible word sequences is extremely large, and only a small portion are seen during training.
     This causes many valid sequences to have zero probability.
   - For examples when computing P(<s> I want to eat Mansaf <\s>) , by Bigram LM, what if "eat Mansaf" didn't appear in the training corpus at all.

$$P(\text{Mansaf} \mid \text{eat}) = \frac{C(\text{eat Mansaf})}{C(w_{i-1})} = \frac{0}{C(w_{i-1})} = 0 \rightarrow P(\text{<s> I want to eat Mansaf <\s>}) = 0$$

2. **Vocabulary limitations**
   - Unseen or rare words are typically mapped to a generic unknown token (OOV), limiting the model's ability to generalize to new input.

Limitations of N-gram LMs

3. Shallow context and lack of long-range dependencies

- N-gram models condition each word only on the previous *(n–1)* words. This short context window limits their understanding of sentence structure and grammar.
- N-gram LM struggle capturing dependencies that span across longer distances, such as subject–verb agreement or clause-level coherence.
- **Example (**Syntactic dependencies**):**
  - In the sentence: "بجدارة **نجحنَ** الطالباتُ المجتهداتُ اللواتي شاركنَ في المسابقة",
  - The verb "**نجحنَ**" must agree with the distant subject "**الطالبات**" in **gender and number**. However, only looks at the previous two words before "**نجحنَ**", and so **cannot** model this grammatical relationship since it “forgets” the original subject.
- **Example (**Semantic dependencies**):**
  - In the sentence: **الولد بجانب العمود بالقرب من متجر البقالة الواقف مع صديقه يبكي بصوت عال.**
  - **Who cries? The boy , the pole , the grocery store, or the friend?**

Handling Unknown N-grams

- Problem: Some sequences are unseen → zero probability.
- **Solutions**:
  - Smoothing
  - Backoff
  - Interpolation
- Goal: Smoothing (or regularizing) parameters to assign some probability mass to unseen events (n-grams).

Rasha Obeidat

Laplace (Add-One) Smoothing

- Add 1 to all counts:
  - **For Bigrams:**

$$p(w_i|w_{i-1}) = \frac{C(w_{i-1},w_i)+1}{C(w_{i-1})+V}$$

    - *V* is the total number of possible bigrams (i.e. the vocabulary size for a bigram model).
  - **For Tri-grams**

$$p(w_i|w_{i-2}\,, w_{i-1}) = \frac{C(w_{i-2,}\, w_{i-1}, w_i) + 1}{C(w_{i-2,}\, w_{i-1}) + V}$$

    - *V* is the vocabulary size for a trigram model).

Limitation of Laplace Smoothing

- Tends to reassign too much mass to unseen events, so can be adjusted to add 0<δ<1 (normalized by δV instead of *V.*
- *Why* Laplace *does that?*
  - **For Bigrams:** $p(w_i|w_{i-1}) = \frac{C(w_{i-1},w_i)+1}{C(w_{i-1})+V}$
- When V is very large (e.g., 100,000 words), the denominator grows a lot.
- This means even frequent observed trigrams shrink because the +1 smoothing is spread across a huge number of possibilities.
- As a result, the true (observed) probabilities get underestimated, while unseen ones get overestimated since both gets very small probabilities . The difference in probabilities between seen and unseen events become small.

Assume:

- $C(w_{i-2}, w_{i-1}) = 100$
- $C(w_{i-2}, w_{i-1}, w_i) = 9$
- Vocabulary size $V = 10{,}000$

Seen trigram (no smoothing):

$$P = \frac{9}{100} = 0.09$$

Seen trigram (Laplace smoothing):

$$P = \frac{9+1}{100+10{,}000} = \frac{10}{10{,}100} \approx 0.00099$$

Unseen trigram (Laplace smoothing):

$$P = \frac{0+1}{100+10{,}000} = \frac{1}{10{,}100} \approx 0.000099$$

Backoff

- If a trigram is unseen, **back off** to bigram.
- If bigram is unseen, **back off** to unigram.
- Always use the longest available history.
- Example:
  - $P(\text{Mansaf}|\text{to, eat})$ missing → use $P(\text{Mansaf}|\text{ eat})$ → if missing, use $P(\text{Mansaf})$

Interpolation

- **Interpolate** between trigram, bigram, and unigram models.

$$P(\text{Mansaf}|\text{to, eat}) = \lambda_1 P(\text{Mansaf}|\text{to, eat}) + \lambda_2 P(\text{Mansaf}|\text{ eat}) + \lambda_3 P(\text{Mansaf})$$

- Assume
  - $P(\text{Mansaf}|\text{to, eat}) = 0.30$
  - $P(\text{Mansaf}|\text{ eat}) = 0.20$
  - $P(\text{Mansaf}) = 0.05$
  - $\lambda_1, \lambda_2, \lambda_3 = 0.6, 0.3, 0.1$ (tuned for lowest ***perplexity*** on dev set)

Perplexity is a metric for evaluating LMs ; **the lower, the better**.

Then,

$$P(\text{Mansaf}|\text{to, eat}) = .6 \times 0.30 + 0.3 \times 0.20 + 0.1 \times 0.05 = 0.245$$

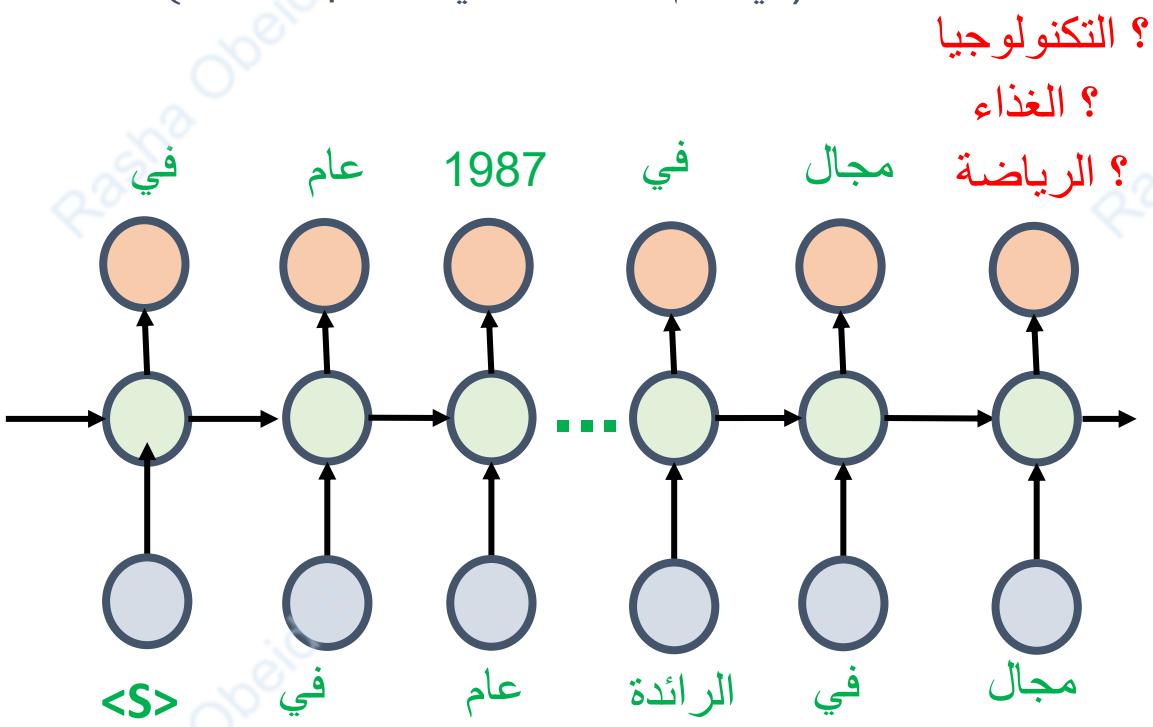
RNN-based Language Model

في عام 1987، أطلقت شركة أبل أول جهاز كمبيوتر محمول لها وقد لاقت منتجاتها اقبالا واسعا. ومنذ ذلك الحين، استمرت الشركة في تطوير منتجاتها. واليوم، تُعد أبل من بين الشركات الرائدة في مجال.....

P(التكنولوجيا | في عام... الرائدة في مجال) ؟

P(الغذاء | في عام... الرائدة في مجال) ؟

P(الرياضة | في عام... الرائدة في مجال) ؟



- RNN learns predict the next word in the sequence given the previous context.
- For example, RNN learns to assign the high probability to "**التكنولوجيا**" than to الغذاء and الرياضة
- However, **vanilla RNNs** suffer from the **vanishing gradient problem**, making them **ineffective at capturing such long-distance dependencies**. As a result, the connection between "**التكنولوجيا**" and " **جهاز كمبيوتر**" might be lost, leading to incorrect predictions.
- Alternatively, LSTM or GRU can be used

- These slides are the intellectual property of **Dr. Rasha Obeidat®**.
- The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
Obeidat.rm@gmail.com
Rmobeidat@just.edu.jo



Rasha Obeidat